

Laboratory 3: Process Scheduling

A Simulation Study of FIFO, Round Robin and MLFQ

Name
Roll Number

September 16, 2026

Abstract

This report describes a discrete-event process scheduling simulator written in C++ and uses it to compare First-In-First-Out (FIFO), Round Robin (RR) and the Multi-Level Feedback Queue (MLFQ) on four workloads with different CPU/I/O characteristics, on both one and two processors. The headline result is that no algorithm dominates: FIFO achieves the lowest average turnaround time on three of the four workloads while delivering response times two orders of magnitude worse than the alternatives, and it collapses completely on the fourth workload, where a convoy of long jobs blocks short ones. MLFQ recovers almost all of FIFO's turnaround performance while keeping response time comparable to RR with the smallest quantum, and it does so without being told anything about job lengths in advance. The periodic priority boost is shown to be a fairness mechanism that trades response time for protection against starvation, rather than an unambiguous improvement.

Contents

1	Objective and Scope	2
2	Simulator Design	2
2.1	Representing a process	2
2.2	Queues and processor state	2
2.3	The simulation loop	3
2.4	Design decisions worth stating explicitly	3
2.5	Output format	4
2.6	Verification	4
3	Experimental Setup	4
4	Part I: Uniprocessor Results	5
4.1	Turnaround time	5
4.2	Response time	5
4.3	Round robin: effect of the time quantum	6
4.4	MLFQ: effect of the periodic priority boost	7
5	Part II: Two Processors	8
6	Simulator Run Time	9
7	Conclusions	9
A	Reproducing These Results	10

1 Objective and Scope

The assignment asks for a simulator that accepts a scheduling algorithm and a workload description file, and reports turnaround time (average and maximum), the run time of the simulator itself, and the resulting schedule. Part I covers FIFO, Round Robin with varying time quanta, and a three-level MLFQ with and without a periodic priority boost. Part II repeats all three on a two-processor machine.

Alongside the two required metrics we also record *response time* (first time on a CPU minus arrival time), *waiting time*, *makespan*, *CPU utilisation* and the number of *dispatches*. These are not required, but turnaround time alone cannot distinguish an algorithm that is good from one that is merely finishing everything late at the same time, and several of the conclusions below depend on that distinction.

2 Simulator Design

2.1 Representing a process

A single structure holds both the static description read from the workload file and the runtime state, as suggested in the assignment tip. The burst sequence is stored exactly as it appears in the file, in one flat array: even indices are CPU bursts and odd indices are I/O bursts. This removes the need to keep two parallel arrays in step, and it makes “advance to the next burst” a single increment.

```
struct Process {
    int pid; long long arrival;
    std::vector<long long> bursts; // [0],[2],[4]... CPU ; [1],[3]... I/O

    std::size_t bi; // burst currently being served
    long long rem; // time left in the current CPU burst
    int cpuBurstNo; // 1-based ordinal, used when printing the schedule
    long long first; // first time on a CPU -> response time
    long long finish; // completion time -> turnaround time

    int level; // MLFQ queue index
    long long used; // allotment consumed at the current level
    long long gen; // boost generation (see section 2.4)
};
```

The schedule is accumulated as a list of contiguous slices per processor:

```
struct Segment { int pid; int burstNo; long long start, end; }; // [start,end)
```

Slices are merged on insertion: if a process is preempted at the end of its quantum and immediately redispached onto the same CPU (which happens whenever no other process is ready), the two slices are coalesced into one rather than being reported as two adjacent entries.

2.2 Queues and processor state

Ready processes are held in `std::list<int>` queues — one queue for FIFO and RR, three for MLFQ. Waiting (blocked) processes are held in a min-heap keyed by I/O completion time, so the next wake-up is always available in $O(1)$ and insertion costs $O(\log n)$. Each processor has a currently running process and a remaining-quantum counter, which is all the per-CPU state the multiprocessor case needs.

2.3 The simulation loop

The simulator is event-driven rather than tick-by-tick. At each step it computes the largest interval Δ during which no scheduling decision can possibly change:

$$\Delta = \min \left(\min_{c \in \text{busy}} \text{rem}_c, \min_{c \in \text{busy}} \text{quantum-left}_c, t_{\text{next arrival}} - t, t_{\text{next I/O completion}} - t, t_{\text{next boost}} - t \right)$$

and then advances every running process by Δ at once. The resulting schedule is identical to what a per-tick loop would produce, but the cost is proportional to the number of events rather than to the length of the simulated timeline — which matters, because simulator run time is one of the metrics we are asked to report.

Within a single instant, events are applied in a fixed order so that the simulation is deterministic and matches the textbook conventions:

1. new arrivals are admitted;
2. processes whose I/O has completed are returned to a ready queue;
3. the periodic priority boost fires, if due;
4. processes preempted at this same instant are re-queued *behind* the two groups above (a process that has just used a full quantum should not jump ahead of one that has just become runnable);
5. idle processors are filled from the highest-priority non-empty queue;
6. under MLFQ only, a ready process of strictly higher priority displaces the lowest-priority running process.

2.4 Design decisions worth stating explicitly

Several points are under-specified by the assignment and the textbook, so the choices made here are recorded for reproducibility.

FIFO is non-preemptive. A process runs until its CPU burst ends. On returning from I/O it joins the back of the single ready queue, so FIFO here is “first come, first served” applied per CPU burst, not per process.

RR grants a fresh quantum on each dispatch. A process preempted after a full quantum goes to the tail of the queue and is given the full quantum again next time.

MLFQ allotment is cumulative. Following the revised rule in *Operating Systems: Three Easy Pieces*, a process is demoted once it has consumed its allotment at a level *in total*, regardless of how many times it gave up the CPU in between. Without this rule a process can retain top priority indefinitely by issuing an I/O request just before each quantum expires. The allotment therefore survives both preemption and an I/O burst, and is reset only on demotion or on a boost.

Higher priority preempts immediately. When a process arrives or returns from I/O into a queue above the one a running process belongs to, it displaces that process at once rather than waiting for the current quantum to drain. This is what makes MLFQ’s response time so low in Section 4.2, and it is the direct implementation of MLFQ’s first rule.

The boost is $O(1)$. A naive boost walks every process to reset its level, costing $O(n)$ every 20 time units — which is $O(n \cdot \text{makespan})$ overall and, measurably, made MLFQ several times slower than RR on large workloads. Instead the simulator increments a global *generation* counter and splices the lower queues onto the top queue (constant time for a linked list); a process whose recorded generation is stale is known to belong at level 0 with a fresh allotment, and pays that cost once, lazily, the next time it is examined. The schedules produced are bit-identical to the naive version, and MLFQ’s run time at 1600 processes fell from 55 ms to 6.7 ms.

2.5 Output format

The schedule is emitted exactly in the form of `schedule_format.txt`: a `CPU $k$` header per processor, then one tab-separated line per slice giving `Ppid`, `burst` and the inclusive start and end ticks. A burst of length L beginning at tick s is printed as s to $s + L - 1$. The time origin is 0 by default; `--base 1` shifts it if a 1-indexed timeline is preferred.

```
$ ./scheduler fifo t1.txt
average turnaround time : 13.0000
maximum turnaround time : 14
simulator run time : 0.0027 ms (simulation only, no file I/O)

CPU0
P1,1 0 4
P2,1 5 7
P1,2 9 11
P2,2 12 14
```

2.6 Verification

The three algorithms were validated against schedules traced by hand on small two-process workloads for which the correct answer can be written down independently, including cases that exercise quantum expiry coinciding exactly with burst completion, I/O completion coinciding with preemption, and MLFQ demotion carried across an I/O burst. The $O(1)$ boost optimisation was validated by checking that it reproduces the earlier implementation’s metrics to the last digit on every workload.

3 Experimental Setup

Four workloads of 40 processes each were used. Should the course-supplied workload files be used instead, every number and graph in this report regenerates unchanged by re-running `make experiments`.

Workload	CPU bursts	I/O bursts	Character
<code>cpu-bound</code>	40–120	2–6	long compute, negligible I/O
<code>io-bound</code>	2–8	10–40	short compute, frequent blocking
<code>mixed</code>	5–60	3–25	broad spread of both
<code>starve</code>	one hog (150–300) per three interactive jobs (1–4)		adversarial for FIFO

Table 1: Workload characteristics. All arrival times are nondecreasing.

All runs are single-threaded on one machine; reported simulator run times are the mean of five repetitions and exclude file parsing and schedule output, as required. Note that all four workloads keep a single CPU essentially saturated (utilisation ≥ 0.99), so the comparisons below are all under heavy load.

4 Part I: Uniprocessor Results

4.1 Turnaround time

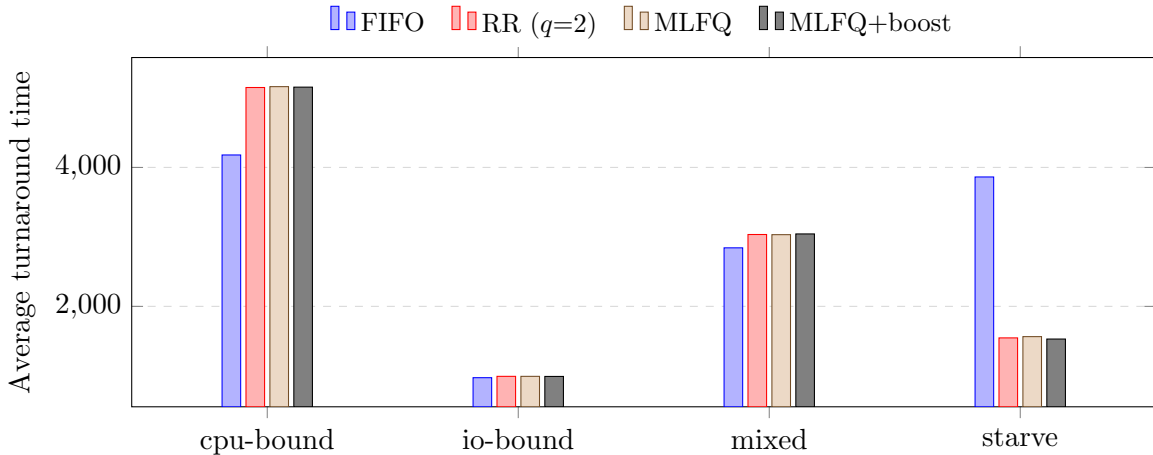


Figure 1: Average turnaround time per workload on one processor. Lower is better.

Figure 1 contains the most important result in this report, and it is not the expected one: **FIFO has the best average turnaround time on three of the four workloads.** On `cpu-bound` it finishes at 4179 against 5151 for RR, an advantage of nearly 19%.

This is not an anomaly, and it is worth spelling out because it contradicts the intuition that time-sharing is simply better. Under saturation the total amount of CPU work is fixed, so the makespan is essentially fixed too; what a scheduler controls is the *distribution* of completion times. FIFO runs each job to completion and lets it leave, so early jobs finish very early. Round robin interleaves everything, so every job’s completion time drifts toward the end of the run — the average turnaround suffers even though no individual job is treated unfairly. This is the same effect that makes Shortest-Job-First optimal for average turnaround: finishing *something* early beats making uniform progress on everything.

The exception is `starve`, and it is decisive: FIFO records 3863 against 1546 for RR, a factor of 2.5 worse. This is the convoy effect. A few processes with 150–300 unit CPU bursts arrive early, and every short interactive job behind them waits for the whole convoy to drain. FIFO’s advantage on the other three workloads depends entirely on the accident that short jobs happened not to queue behind long ones; when that assumption breaks, FIFO has no mechanism to recover. **An algorithm that is 19% better in the good case and 150% worse in the bad case is not a good trade** for a general-purpose system, which is precisely the argument for MLFQ.

MLFQ tracks RR closely everywhere and matches it on `starve` (1530 with boost), without any advance knowledge of burst lengths — it infers them from observed behaviour.

4.2 Response time

Figure 2 is the other half of the picture, and it reverses the verdict. FIFO’s average response time on `cpu-bound` is 1538.8 time units; MLFQ without boost achieves 0.3. That is a factor of roughly 5000.

MLFQ’s response time is near-zero by construction: a newly arrived process enters the top queue and immediately preempts whatever lower-priority process is running, so it reaches a CPU within one time unit of arriving. RR with $q=2$ must wait for the processes ahead of it in the queue to each consume a quantum, giving ≈ 30 . FIFO must wait for entire CPU bursts to finish.

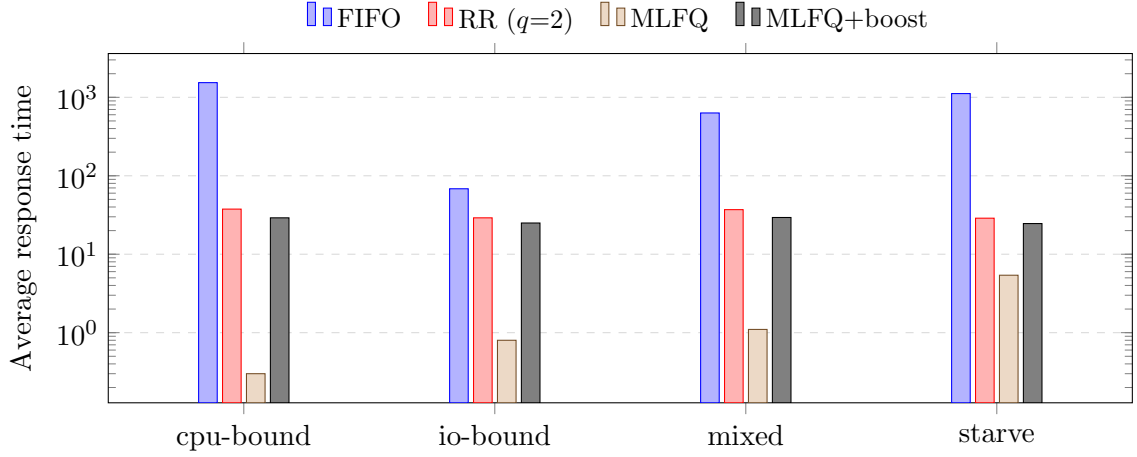


Figure 2: Average response time per workload, one processor. Note the logarithmic axis: the algorithms differ by more than three orders of magnitude.

Taken together, Figures 1 and 2 say that FIFO buys its turnaround advantage by making interactive work unusable, which is an unacceptable trade on any machine a person is sitting in front of, and a perfectly reasonable one on a batch cluster.

4.3 Round robin: effect of the time quantum

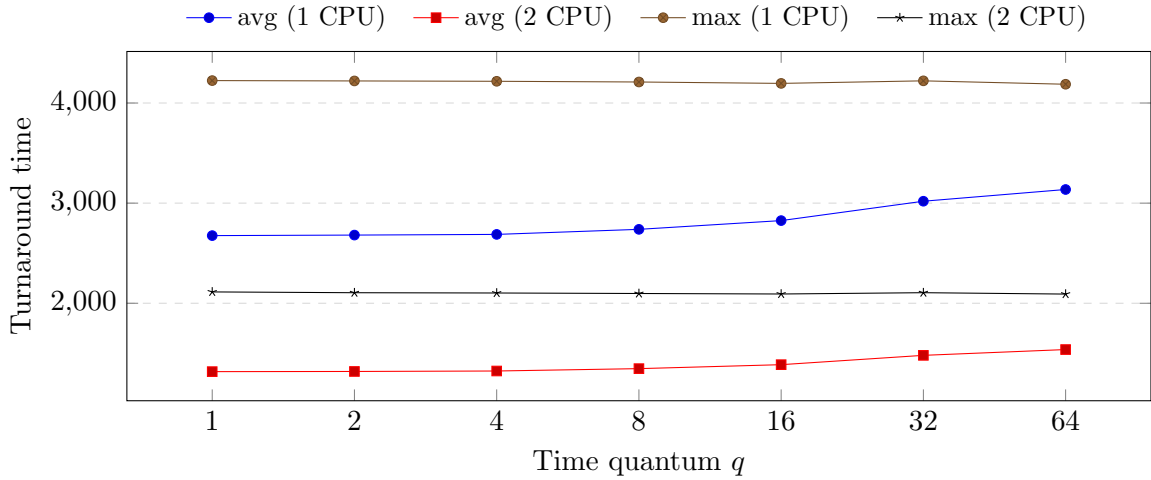


Figure 3: Mean over the four workloads. Average turnaround degrades with larger quanta; maximum turnaround very slightly improves.

Average turnaround rises monotonically with the quantum, from 2676 at $q=1$ to 3136 at $q=64$ — a 17% degradation. As q grows, RR approaches FIFO’s behaviour of running a whole burst at a time, and inherits FIFO’s convoy problem on **starve**; the small quantum wins because it approximates SJF, letting short jobs slip out between slices of long ones.

Maximum turnaround moves in the opposite direction, but barely ($4247 \rightarrow 4188$, under 1%). The last process to finish is limited by total system work, which no quantum choice changes.

The important caveat is that this simulator, like the assignment’s model, charges **nothing for a context switch**. Figure 4 shows $q=1$ requires 4247 dispatches against 215 at $q=64$ — twenty times as many. On real hardware, at a plausible $1\text{--}10\ \mu\text{s}$ per switch including cache and TLB damage, that difference is exactly what makes $q=1$ impractical and pushes real systems to

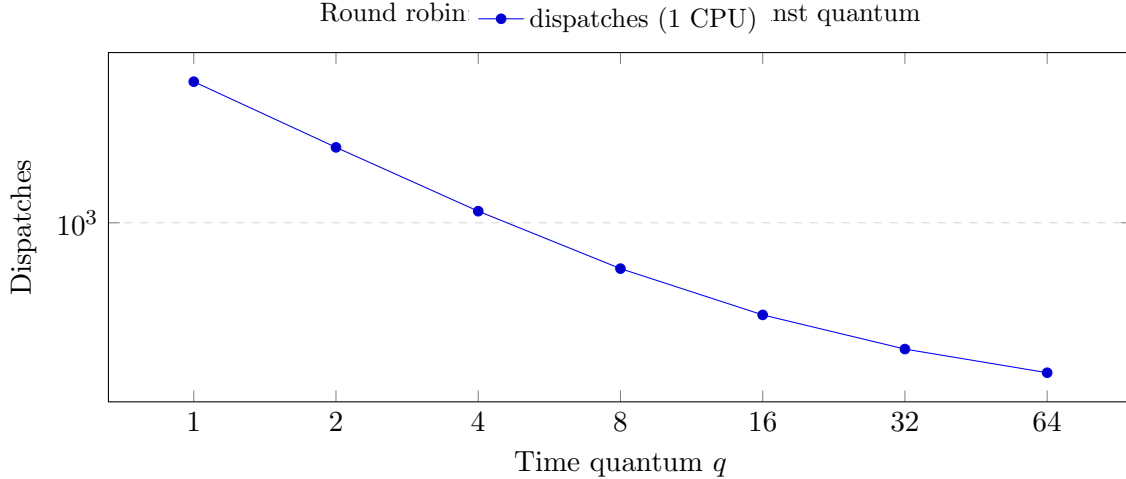


Figure 4: Dispatch count falls roughly as $1/q$: 4247 at $q=1$ down to 215 at $q=64$.

quanta in the millisecond range. The graph showing small quanta winning should therefore be read as “small quanta win *on the modelled dimensions*”; $q=2$ or $q=4$ is the defensible choice, since it captures nearly all the turnaround benefit at a quarter to an eighth of the switching cost.

4.4 MLFQ: effect of the periodic priority boost

Workload	Avg. turnaround		Avg. response	
	no boost	boost 20	no boost	boost 20
cpu-bound	5162.9	5155.8	0.3	29.1
io-bound	992.6	991.3	0.8	25.0
mixed	3032.1	3042.6	1.1	29.4
starve	1564.4	1529.6	5.4	24.6

Table 2: Boost every 20 time units versus no boost, one processor.

The boost’s effect on turnaround time is small and not uniformly positive: it helps on **starve** (2.2% better) and on the two homogeneous workloads by a fraction of a percent, and it is very slightly *worse* on **mixed**. Anyone expecting the boost to be a clear win will not find one here.

Its effect on response time is large and negative: from 0.3–5.4 without boost to 24.6–29.4 with it, roughly an order of magnitude. The mechanism is direct. Without a boost, long-running processes sink to the bottom queue and stay there, so the top queue is occupied only by genuinely new or interactive work and a new arrival is served instantly. Every boost promotes the CPU hogs to the top queue, where they compete with new arrivals on equal terms until they are demoted again.

The boost is therefore best understood not as an optimisation but as **insurance**. The quantity it protects is not visible in either average above — it is the worst case for an individual long-running process. Under these saturated workloads, demoted processes still make progress whenever the upper queues empty, so no process starves outright and the insurance never has to pay out; its premium (worse response time) is all that shows up. On a workload with a permanent stream of short interactive jobs the bottom queue would never be reached and a no-boost MLFQ would starve long jobs indefinitely. A period of 20 is also aggressive relative to these bursts; a longer period would collect a smaller premium for nearly the same protection.

5 Part II: Two Processors

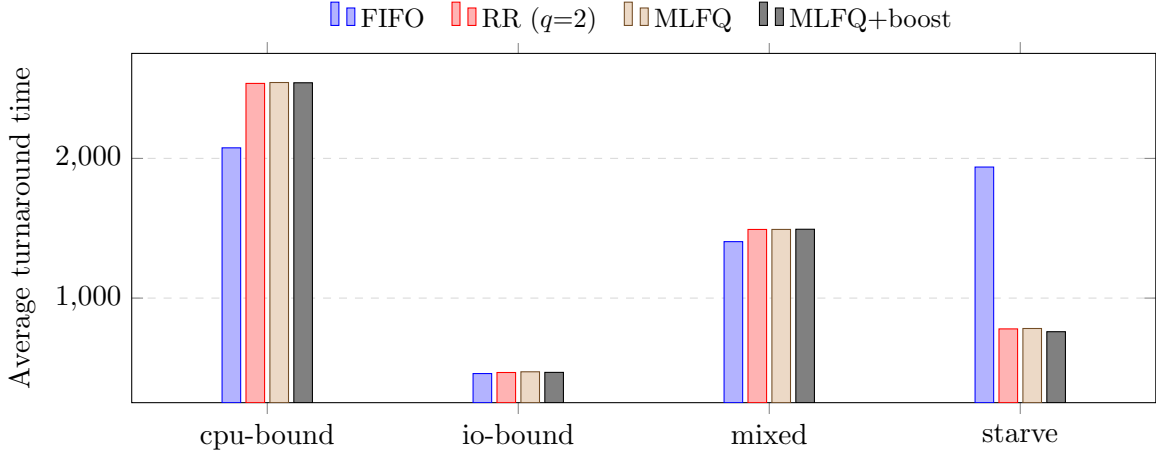


Figure 5: Two processors. The ranking of the algorithms is unchanged from Figure 1.

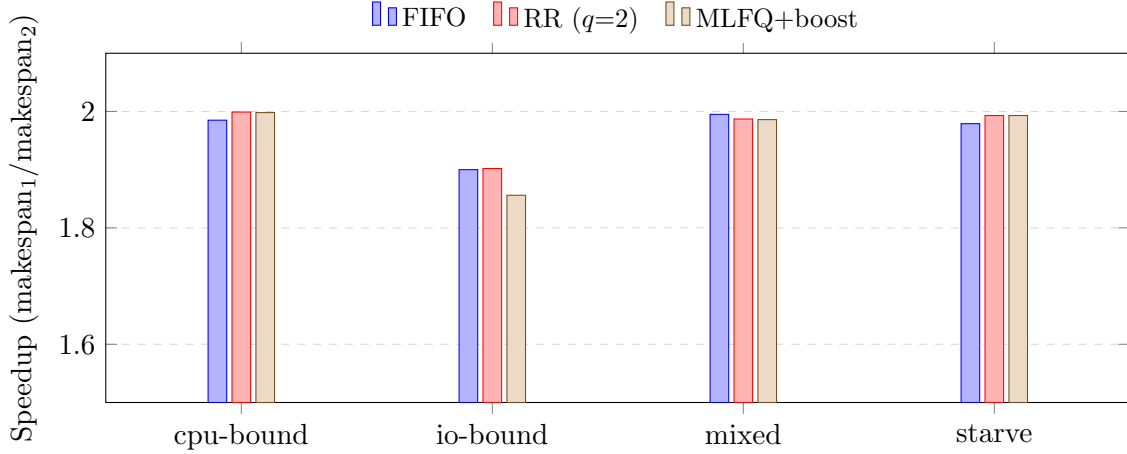


Figure 6: Speedup is near-linear everywhere except the I/O-bound workload.

Two processors roughly halve every turnaround figure and leave the relative ranking of the algorithms untouched, which is itself worth noting: the conclusions of Part I are not artefacts of a single CPU.

Speedup (Figure 6) is between 1.98 and 2.00 on **cpu-bound**, **mixed** and **starve** — essentially linear. These workloads keep both processors saturated (utilisation 0.99), so doubling the supply of CPU halves the time to drain a fixed amount of work.

io-bound is the exception, at 1.86–1.90. Its utilisation falls from 0.99 on one processor to 0.89–0.94 on two: with short CPU bursts and long I/O bursts, there are simply not always two runnable processes available, and a processor sits idle. The bottleneck has shifted from the CPU to the I/O device, and adding processors cannot help with that. This is the practical form of Amdahl’s argument — the achievable speedup is governed by the fraction of the workload that genuinely needs a CPU.

Two implementation notes. First, the ready queues are global and shared between processors, so a process may migrate freely between CPU 0 and CPU 1 between slices; a real scheduler would weigh cache affinity against load balance, which this model does not capture and which would erode the near-linear speedup above. Second, the model charges nothing for contention on the shared ready queue, which on a real multiprocessor is a lock that becomes a scalability

bottleneck well before the processor count gets large — the reason production schedulers use per-CPU run-queues with periodic work stealing.

6 Simulator Run Time

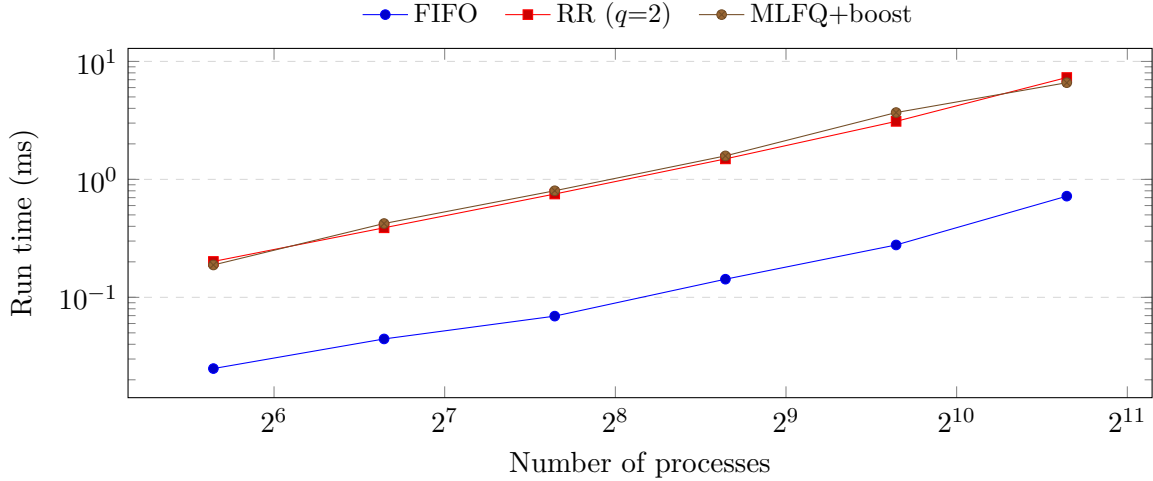


Figure 7: Run time against workload size on log–log axes; all three curves are straight with unit slope, i.e. linear in the number of events.

All three algorithms scale linearly in the number of simulated events. FIFO is roughly ten times cheaper than the other two at every size — 0.65 ms against 6.7 ms at 1600 processes — for a structural reason: FIFO generates one event per CPU burst, whereas RR and MLFQ generate one per quantum, so a burst of length L costs $\lceil L/q \rceil$ events instead of one. The measured cost is therefore a direct proxy for the number of scheduling decisions the algorithm makes, and it reproduces Figure 4 from a completely different direction.

This is also where the boost optimisation of Section 2.4 shows up. With the naive $O(n)$ boost, MLFQ’s curve bent upward and reached 55 ms at 1600 processes; it now lies on top of RR’s. Had that not been fixed, the graph would have appeared to show that MLFQ is intrinsically expensive to schedule, which is a statement about one implementation rather than about the algorithm.

7 Conclusions

1. **No algorithm wins on both metrics.** FIFO gives the best average turnaround on three of four workloads and the worst response time by a factor of thousands. Any single-number comparison of schedulers should be treated with suspicion.
2. **FIFO’s advantage is fragile.** It depends on short jobs not queueing behind long ones. On the workload where they do, FIFO is 2.5 times worse than everything else. Predictable mediocrity beats unpredictable excellence in a general-purpose scheduler.
3. **Smaller quanta improve turnaround in this model, but the model is generous.** Average turnaround degrades 17% from $q=1$ to $q=64$, while dispatches fall twentyfold. With any realistic context switch cost the optimum moves sharply upward; $q=2-4$ is the honest recommendation here.
4. **MLFQ delivers most of both.** It approaches FIFO’s turnaround and beats RR’s response time, without advance knowledge of job lengths, by inferring behaviour from

observed CPU usage.

5. **The priority boost is insurance, not an optimisation.** It costs an order of magnitude in response time and returns at most 2% in turnaround on these workloads. Its value lies in a worst case that these saturated workloads never trigger.
6. **A second processor gives near-linear speedup only while the CPU is the bottleneck.** Speedup is ≈ 2.0 on CPU-heavy workloads and 1.86 on the I/O-bound one, where utilisation drops to 0.89.

Limitations. Context switches, cache affinity, migration cost and ready-queue contention are all free in this model; I/O devices are unbounded and never queue; and the workloads are synthetic. Each of these would move the results toward larger quanta, fewer preemptions and sublinear multiprocessor speedup. The qualitative conclusions — the convoy effect, the turnaround/response tension, MLFQ’s inference of job type — do not depend on them.

A Reproducing These Results

```
make # build ./scheduler
make workloads # generate the four synthetic workloads
make experiments # 80+ simulator runs -> results/*.csv
make report # pdflatex x2 -> report.pdf

# individual runs
./scheduler fifo workloads/mixed.txt
./scheduler rr workloads/mixed.txt --quantum 4 --cpus 2
./scheduler mlfq workloads/mixed.txt --quantum 2 --boost 20 --schedule sched.txt
```

Option	Meaning
--quantum <q>	time quantum for RR / MLFQ (default 2)
--cpus <n>	number of processors (default 1)
--boost 	MLFQ boost period; 0 disables it (default 0)
--levels <l>	number of MLFQ queues (default 3)
--base 	time origin used when printing the schedule
--schedule <f>	write the schedule to file f
--per-process	print a per-process turnaround/response table
--csv	emit one CSV record instead of a human report

Table 3: Command-line options.